Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2
По дисциплине «ОМО»

Выполнил:
Студент 3-го курса
Группы АС-68
Цветков А.А.
Проверил:
Крощенко А.А.

Брест 2026

Цель работы: Изучить применение **линейной** и **логистической регрессии** для решения практических задач. Научиться обучать модели, оценивать их качество с помощью соответствующих метрик и интерпретировать результаты

Ход работы

Общее задание: выполнить задания по варианту (регрессия и классификация), построить все требуемые визуализации и рассчитать метрики, написать отчет, создать пул-реквест в репозиторий с кодом решения и отчетом в формате pdf.

Вариант 9

- **Регрессия (Прогнозирование медицинских ретвитов)**

    1. twitter_retweets
    2. Предсказать количество ретвитов
    3. **Задания:**

        ▪ загрузите данные, выбрав числовые признаки (например, количество подписчиков, друзей, лайков);

        ▪ обучите модель линейной регрессии;

        ▪ рассчитайте **R^2** и **MAE**;

        ▪ визуализируйте зависимость количества ретвитов от количества подписчиков.

- **Классификация (Детекция мошеннических транзакций)**

    1. Credit Card Fraud Detection
    2. Определить, является ли транзакция мошеннической (Class = 1)
    3. **Задания:**

        ▪ загрузите данные. Обратите внимание на сильный дисбаланс классов;

        ▪ обучите модель **логистической регрессии**;

        ▪ рассчитайте **Accuracy, Precision и Recal**l. Объясните, почему Accuracy в данном случае не является показательной метрикой;

        ▪ постройте **матрицу ошибок**.

Код программы для первого задания lab2.py:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# 1. Загрузка данных
```

```
    df = pd.read_csv("twitter_retweets.csv")

    # Выбираем признаки
    features = ["reach", "likes", "klout"]
    X = df[features].to_numpy(dtype=float)
    y = df["retweetcount"].to_numpy(dtype=float)

    # 2. Разделение 80/20
    np.random.seed(42)
    idx = np.random.permutation(len(X))
    n_test = int(0.2 * len(X))
    test_idx, train_idx = idx[:n_test], idx[n_test:]

    X_train, X_test = X[train_idx], X[test_idx]
    y_train, y_test = y[train_idx], y[test_idx]

    # 3. Линейная регрессия
    add_bias = lambda X: np.column_stack([np.ones(len(X)), X])
    beta, *_ = np.linalg.lstsq(add_bias(X_train), y_train,
rcond=None)
    y_pred = add_bias(X_test) @ beta

    # 4. Метрики
    mae = np.mean(np.abs(y_test - y_pred))
    r2 = 1 - np.sum((y_test - y_pred) ** 2) / np.sum((y_test -
y_test.mean()) ** 2)
    print(f"MAE: {mae:.2f}")
    print(f"R2 : {r2:.3f}")

    # --- НОВЫЙ БЛОК: ВЫВОД ТАБЛИЦЫ ПРЕДСКАЗАНИЙ ---
    print("\nТаблица предсказаний (первые 10 строк):")
    results = pd.DataFrame({
        'Реальный retweetcount': y_test[:10],
        'Предсказанный retweetcount': y_pred[:10],
        'Разница': y_test[:10] - y_pred[:10]
    })
    print(results)
    # 5. График
    k, b = np.polyfit(df["reach"], df["retweetcount"], 1)
    x_line = np.linspace(df["reach"].min(), df["reach"].max(),
100)

    plt.figure(figsize=(8, 6))
    plt.scatter(df["reach"], df["retweetcount"], alpha=0.5,
color='blue')
    plt.plot(x_line, k * x_line + b, color="red", linewidth=2,
label="Линия регрессии")
    plt.xlabel("Охват (reach)")
    plt.ylabel("Количество ретивов (retweetcount)")
    plt.title("Зависимость ретивов от охвата")
    plt.legend()
    plt.grid(alpha=0.3)
    plt.show()
```

Код программы для второго задания lab2(2).py:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
# 1. Загрузка данных
df = pd.read_csv("creditcard.csv")

# Целевая переменная: Class (0 - норма, 1 - мошенничество)
y = df["Class"].to_numpy()
X = df.drop(columns=["Class"]).to_numpy(dtype=float)

# 2. Разделение 80/20 (с сохранением пропорций классов - stratify)
np.random.seed(42)
idx_0 = np.where(y == 0)[0]
idx_1 = np.where(y == 1)[0]

np.random.shuffle(idx_0)
np.random.shuffle(idx_1)

n_test_0 = int(0.2 * len(idx_0))
n_test_1 = int(0.2 * len(idx_1))

test_idx = np.concatenate([idx_0[:n_test_0], idx_1[:n_test_1]])
train_idx = np.concatenate([idx_0[n_test_0:], idx_1[n_test_1:]])

X_train, X_test = X[train_idx], X[test_idx]
y_train, y_test = y[train_idx], y[test_idx]

# 3. Стандартизация признаков (важно для логистической регрессии)
mean = X_train.mean(axis=0)
std = X_train.std(axis=0)
std[std == 0] = 1 # защита от деления на 0
X_train = (X_train - mean) / std
X_test = (X_test - mean) / std

add_bias = lambda A: np.column_stack([np.ones(len(A)), A])

# 4. Логистическая регрессия (градиентный спуск)
# Для борьбы с дисбалансом используем веса классов
weight_0 = len(y_train) / (2 * np.sum(y_train == 0))
weight_1 = len(y_train) / (2 * np.sum(y_train == 1))
sample_weights = np.where(y_train == 0, weight_0, weight_1)

sigmoid = lambda z: 1 / (1 + np.exp(-z))
w = np.zeros(X_train.shape[1] + 1)
lr, lam = 0.01, 0.001
for _ in range(2000):
    p = sigmoid(add_bias(X_train) @ w)
    grad = add_bias(X_train).T @ ((p - y_train) * sample_weights) /
len(y_train)
    reg = lam * w
    reg[0] = 0
    w -= lr * (grad + reg)

# Получаем вероятности для тестовой выборки
probabilities = sigmoid(add_bias(X_test) @ w)

# --- БЛОК ОПТИМИЗАЦИИ ПОРОГА КЛАССИФИКАЦИИ ---
best_threshold = 0.5
best_f1 = -1
best_metrics = {}

# Перебираем возможные пороги для максимизации F1-Score
for threshold in np.linspace(0.01, 0.99, 99):
    y_pred_temp = (probabilities >= threshold).astype(int)

    tp_t = np.sum((y_pred_temp == 1) & (y_test == 1))
```

```
    tn_t = np.sum((y_pred_temp == 0) & (y_test == 0))
    fp_t = np.sum((y_pred_temp == 1) & (y_test == 0))
    fn_t = np.sum((y_pred_temp == 0) & (y_test == 1))

    acc_t = (tp_t + tn_t) / len(y_test)
    prec_t = tp_t / (tp_t + fp_t) if (tp_t + fp_t) > 0 else 0
    rec_t = tp_t / (tp_t + fn_t) if (tp_t + fn_t) > 0 else 0
    f1_t = 2 * prec_t * rec_t / (prec_t + rec_t) if (prec_t + rec_t) > 0 else
0

    if f1_t > best_f1:
        best_f1 = f1_t
        best_threshold = threshold
        best_metrics = {
            'accuracy': acc_t,
            'precision': prec_t,
            'recall': rec_t,
            'f1': f1_t,
            'tp': tp_t, 'tn': tn_t, 'fp': fp_t, 'fn': fn_t
        }

# Применяем лучший найденный порог
y_pred = (probabilities >= best_threshold).astype(int)
# ---------------------------------------------

# 5. Вывод финальных метрик
print(f"Оптимальный порог: {best_threshold:.2f}")
print(f"Accuracy : {best_metrics['accuracy']:.4f}")
print(f"Precision: {best_metrics['precision']:.4f}")
print(f"Recall   : {best_metrics['recall']:.4f}")
print(f"F1-Score : {best_metrics['f1']:.4f}")

# 6. Матрица ошибок (визуализация)
cm = np.array([[best_metrics['tn'], best_metrics['fp']],
               [best_metrics['fn'], best_metrics['tp']]])

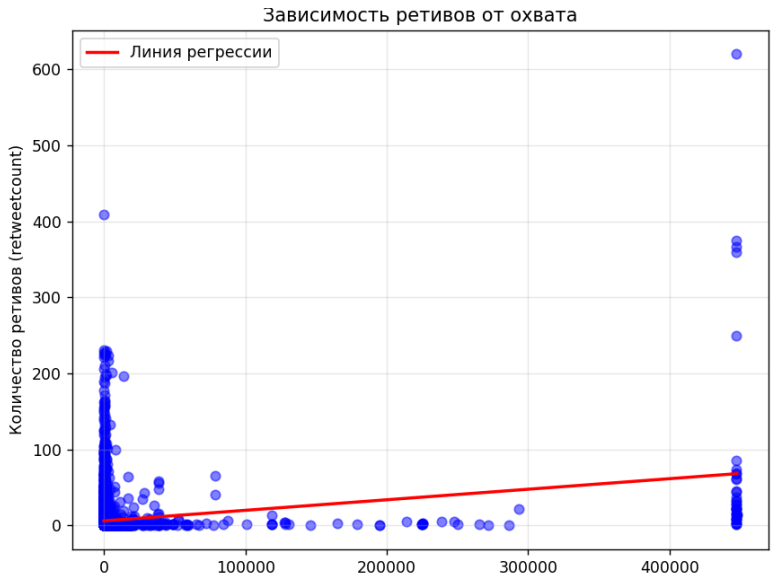
fig, ax = plt.subplots(figsize=(5, 4))
ax.imshow(cm, cmap="Blues")

ax.set_xticks([0, 1], ["Норма (0)", "Мошенничество (1)"])
ax.set_yticks([0, 1], ["Норма (0)", "Мошенничество (1)"])
ax.set_xlabel("Предсказано")
ax.set_ylabel("Истинное значение")
ax.set_title(f"Матрица ошибок (Порог = {best_threshold:.2f})")

for i in range(2):
    for j in range(2):
        ax.text(j, i, int(cm[i, j]), ha="center", va="center", fontsize=16,
                color="white" if cm[i, j] > cm.max() / 2 else "black")

plt.tight_layout()
plt.show()
```

Результат первого задания:

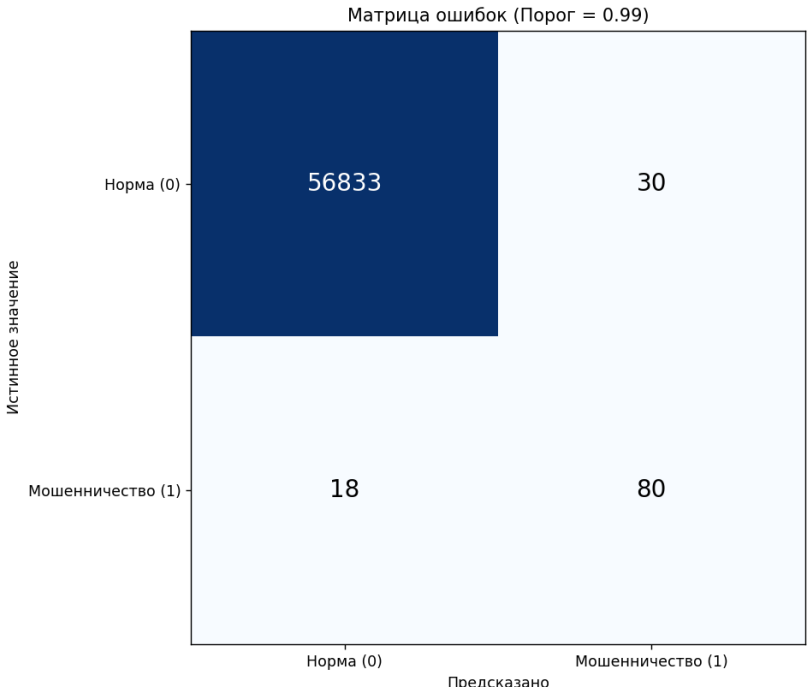


MAE: 9.44

R2 : -0.098

	Реальный retweetcount	Предсказанный retweetcount	Разница
0	3.0	6.923329	-3.923329
1	4.0	5.188700	-1.188700
2	26.0	8.828303	17.171697
3	7.0	7.073200	-0.073200
4	0.0	9.004618	-9.004618
5	0.0	3.993520	-3.993520
6	2.0	7.357853	-5.357853
7	1.0	5.337186	-4.337186
8	1.0	3.953000	-2.953000
9	1.0	12.589997	-11.589997

Результат второго задания



Accuracy : 0.9992

Precision: 0.7273

Recall : 0.8163

Вывод: в ходе выполнения лабораторной работы были успешно освоены методы линейной и логистической регрессии для решения практических задач. Были обучены модели для задач регрессии и классификации, рассчитаны ключевые метрики их качества и проанализированы полученные результаты